USTO-MB – Faculty of Mathematics and Informatics / **Department of Computer Science**
Master's Degree Program : **Artificial Intelligence, Data Science, Networks and Distributed Systems**
Master : Artificial Intelligence, Data Science, Networks and Distributed Systems
Subject : Advanced Algorithms and Complexity / 2026 – 2027

Lab Sheet — TP 4

Text Algorithms in Practice

Corresponds to: Lecture 5 (see TD 4)

Session Objectives

- Implement naive string matching and the KMP algorithm.
- Compare their running time on a repetitive text.
- Apply pattern matching to a small text-processing task.

Exercise 1 — Guided (done together in session)

Complete the naive string-matching function, returning all starting positions where the pattern occurs in the text.

```
def naive_search(text, pattern):
    n, m = len(text), len(pattern)
    positions = []
    for i in range(n - m + 1):
        # TODO: check if text[i:i+m] == pattern, and if so append i
        pass
    return positions
assert naive_search('abcabcabc', 'abc') == [0, 3, 6]
print('OK')
```

Exercise 2 — Autonomous (in session, self-checked with the tests provided)

Complete the KMP prefix-function and search functions below, and check that they return the same matches as naive_search on several test strings.

```
def compute_prefix(pattern):
    m = len(pattern)
    pi = [0] * m
    k = 0
    for i in range(1, m):
        while k > 0 and pattern[i] != pattern[k]:
            k = pi[k - 1]
        if pattern[i] == pattern[k]:
            k += 1
        pi[i] = k
    return pi

def kmp_search(text, pattern):
    if not pattern:
        return []
    pi = compute_prefix(pattern)
    n, m = len(text), len(pattern)
    k = 0
    positions = []
    for i in range(n):
        # TODO: while k > 0 and text[i] != pattern[k], fall back using pi
        # TODO: if text[i] == pattern[k], advance k
        # TODO: if k == m, a match starts at i - m + 1; record it and fall back using pi
    return positions

for t, pat in [('abcabcabc', 'abc'), ('aaaaaa', 'aa'), ('bacbababaabcbababaca', 'ababaca')]:
    assert kmp_search(t, pat) == naive_search(t, pat), (t, pat)
print('KMP matches naive_search on all tests')
```

Exercise 3 — Optional, to take further at home

A short synthetic "DNA" sequence is a string over the alphabet {A, C, G, T}. Generate a random sequence of 100 000 characters, plant a specific 12-character motif at 5 random positions, and use kmp_search to recover all occurrences (the planted ones, plus possibly a few that occur by chance).